PORTFOLIO PROJECT -- README

Job Tracker

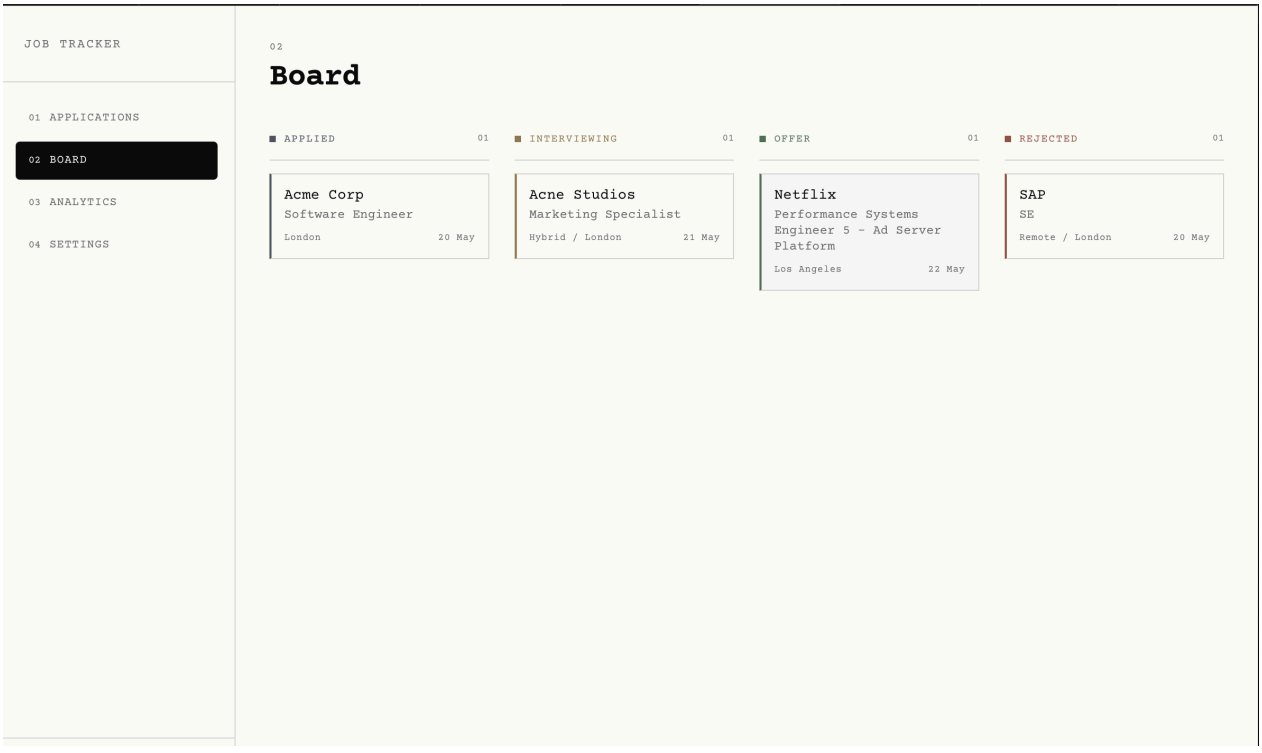
A job application tracker built to be used, not just demoed

Links

Live demo — `[ url ]`
Repository — `[ url ]`
Project write-up — `[ url ]`

Stack

Next.js 14, Supabase
Tailwind + shadcn/ui
DistilBERT, Modal
Deployed on Vercel



1 Why This Exists

I was tracking job applications in a spreadsheet and it was doing the job badly. No sense of pipeline, and a lot of repetitive data entry every time I found a posting. So I built the tool I actually wanted. Something that shows the shape of a job search and removes the tedious parts of logging an application.

It's also a portfolio project, so a second goal ran alongside the first. Build something that demonstrates real full-stack and my own introduction to ML work.

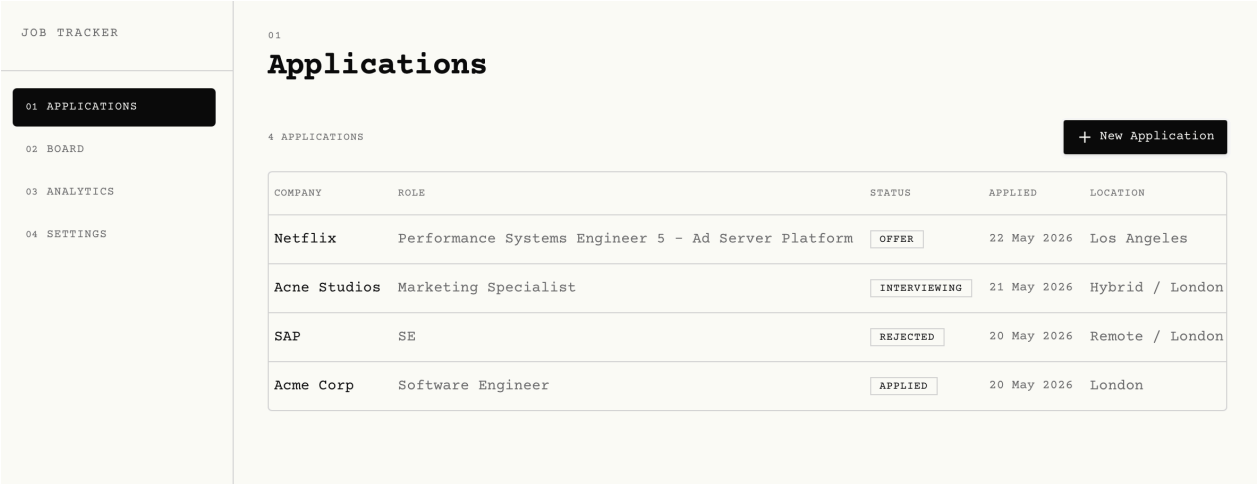
2 Stack

- **Next.js 14** (App Router): server components, server actions
- **Supabase**: Postgres, authentication, RLS
- **Tailwind CSS + shadcn/ui**: styling and component primitives
- **DistilBERT** (fine-tuned): named entity extraction for the URL parser
- **Modal**: stz hosting for the ML inference endpoint
- **Vercel**: deployment

3 Features

3.1 Applications (table view)

Every application in a sortable table, which looks like this; company, role, status, applied date, location. Add and edit through a dialog with the full field set (salary range, job URL, notes). Delete asks for confirmation. A saved job URL opens in a new tab. Proper empty and loading states throughout.

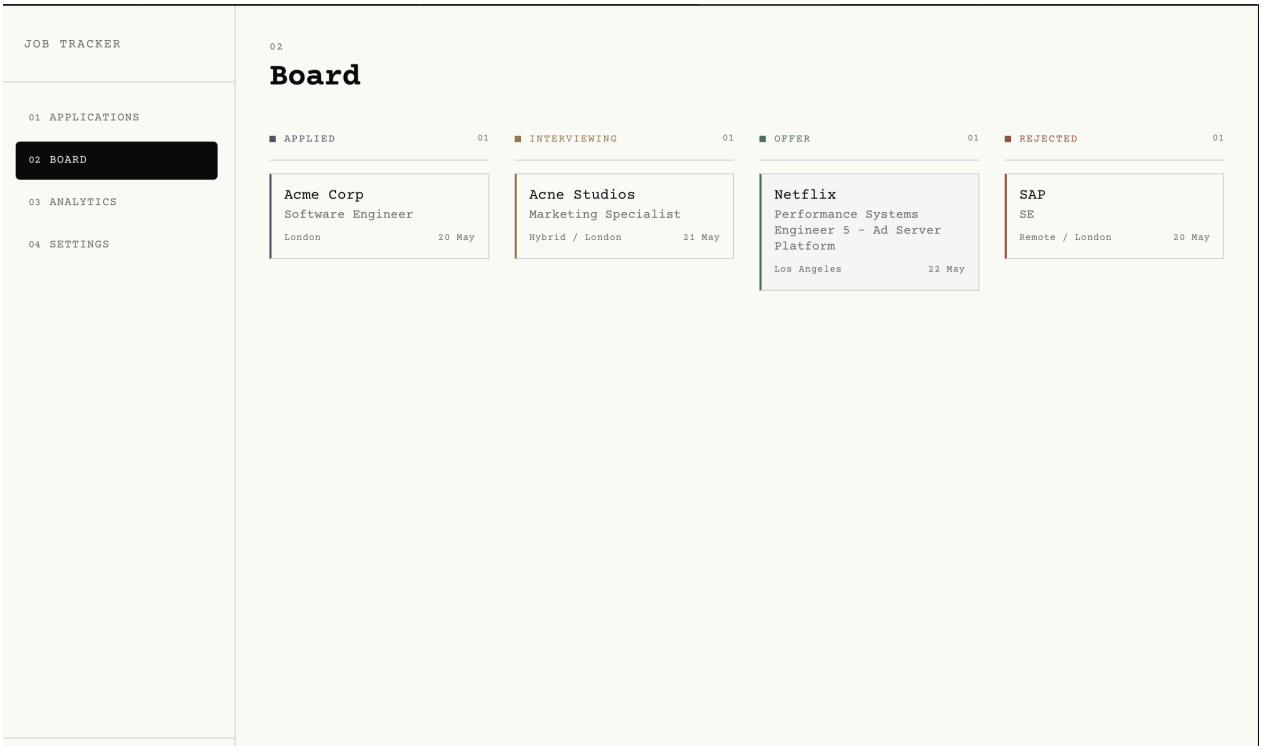


3.2 Board (kanban view)

Four columns: Applied, Interviewing, Offer, Rejected. Cards drag between columns and their position persists. The UI updates, so the card moves immediately and only rolls back if the server rejects the change.

The board is the one view with colour. Each column carries a muted hue, graphite, ochre, sage, rust, applied as a 2px left border on cards and an accent on the column header. The hues were

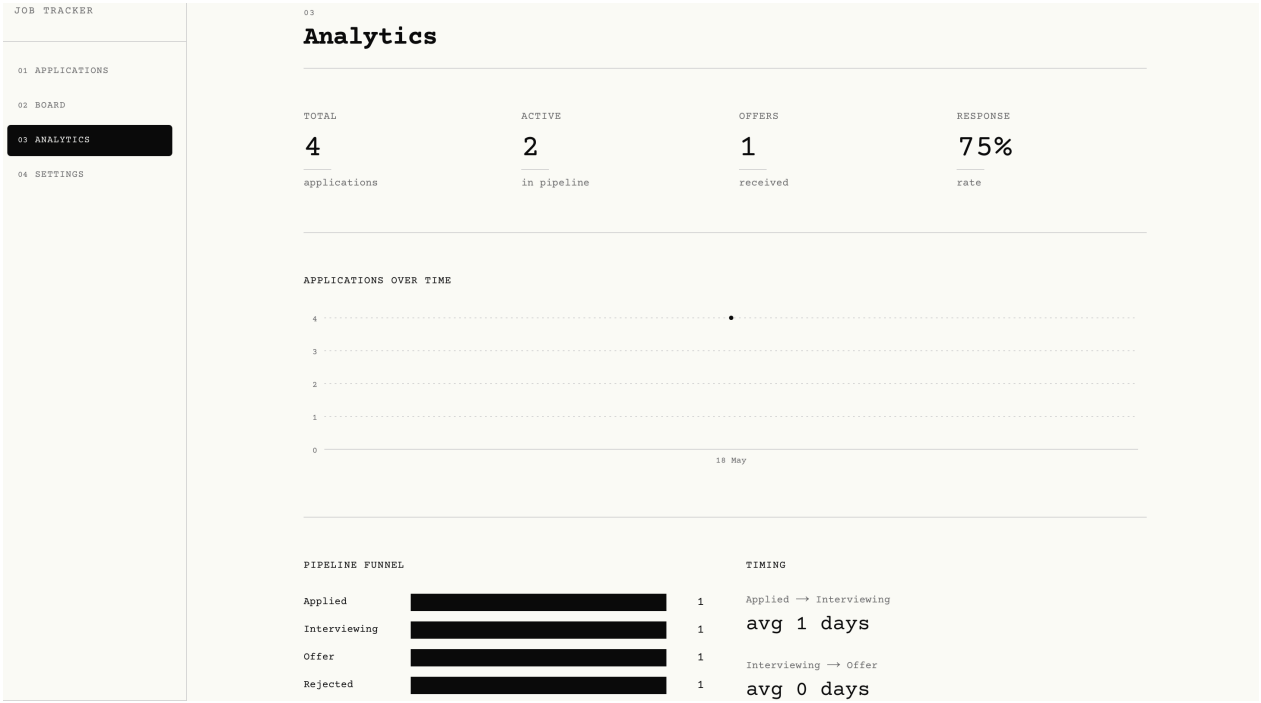
chosen for editorial neutrality rather than traffic-light convention. Sage for offer instead of bright green, rust for rejected instead of red. Every other view stays black-and-white, which is what makes the colour on the board read as deliberate.



3.3 Analytics

A dashboard that answers five questions: how many applications, how many active in the pipeline, how many offers, application velocity, and stage timing. A weekly line chart, a proportional funnel, and an average-days breakdown between stages.

All the aggregation happens in Postgres via a view and two RPC functions, not in application code. The database is the right layer for this work. Faster, and it keeps the query logic in one place.



3.4 AI job parser

The feature I'm most pleased with. The new-application dialog has a "quick fill" row: paste a job URL, and the form pre-fills company, role, location, and salary.

The pipeline does the cheap thing first. For Greenhouse and Lever URLs, and for any page that exposes JSON-LD `JobPosting` structured data, the fields are extracted directly from the HTML with CSS selectors, no model involved. The fine-tuned model is the fallback for everything else. A toast tells the user which path ran, so the tool is upfront about when it is guessing.

LinkedIn, Indeed, and Workday are explicitly unsupported. They either block scrapers or require JS rendering. These return a clear error rather than failing silently.

[GIF: paste a URL, watch the fields populate]

3.5 Settings and dark mode

Profile and password management, a default-view preference, JSON and CSV export, and a danger zone for deleting data or the whole account (destructive actions require typing a confirmation phrase). Full dark mode, warm off-white on near-black, designed rather than auto-inverted, that syncs to the database so it follows the user accross devices.

4 The Machine Learning Piece

The URL parser's fallback is a DistilBERT model fine-tuned for token classification. It identifies five field types (company, role, location, salary, employment type) using BIO tagging across 11 label classes.

4.1 Dataset

Training data was synthetic: 800 job postings generated with Claude, with the field values fixed at generation time so the labels were known by construction. This sidesteps the main cost of an NER dataset, which is manual span labelling, while keeping the labels exact.

The validation set was real: 72 job postings hand-scraped from Greenhouse and labelled by hand. Synthetic data is only useful if it generalises to real postings, and the only way to know that is to measure against real ones. 72 is a small set and I would grow it if I took this further, but it was enough to show the model wasn't just memorising the synthetic distribution.

4.2 Deployment

The model runs on Modal as a scale-to-zero FastAPI endpoint. No cost when idle, a cold start of a few seconds on the first request, sub-two-second inferance once warm. The Next.js app calls it server-side and the call is rate-limited per user.

5 Security

Security was treated as a feature. The notable decisions:

Authorization lives in the database. RLS is enabled on every table, so the database itself

enforces that a user can only ever read or write their own rows. A bug in the application layer can't leak another user's data.

RPC functions respect RLS. The analytics aggregation functions use `security invoker`, so they run with the caller's permissions and RLS still applies. The one exception is the profile-creation trigger, which uses `security definer` because it runs during signup, before the user is authenticated.

Input is never trusted. Every server action validates its input with Zod before touching the database. URL fields are validated as URLs, which rejects `javascript:` pseudo-URLs, otherwise an XSS vector on the "open job listing" button.

Rate limiting is identifier-aware. Auth endpoints are limited by IP because the user isn't known yet. Authenticated actions are limited by user ID. Limiting per-user actions by IP would let one user on a shared network accidently rate-limit everyone else on it.

Secrets stay server-side. The admin client used for account deletion imports the `server-only` package, so the build fails if it ever gets pulled into client code. `gitleaks` was run over the history before deploying.

HTTP security headers are set on every route. Custom 404 and error pages replace the default Next.js ones.

6 What I Would Do Differently

A few things I'd change:

- **Stage timing is approximate.** The analytics "average days between stages" is computed from `applied_at` and `updated_at`, because the schema doesn't log per-stage transitions. A `status_history` table would make this exact. It was a deliberate tradeoff, but it's the clearest piece of technical debt in the project.
- **The salary field underperforms**, for the reasons in the ML section above.
- **The validation set is small** at 72 examples. Enough to be indicative, not enough to be precise.

None of these block the tool from being genuinely useful, which was the bar I set.

Canonical version: `README.md` in the repository.